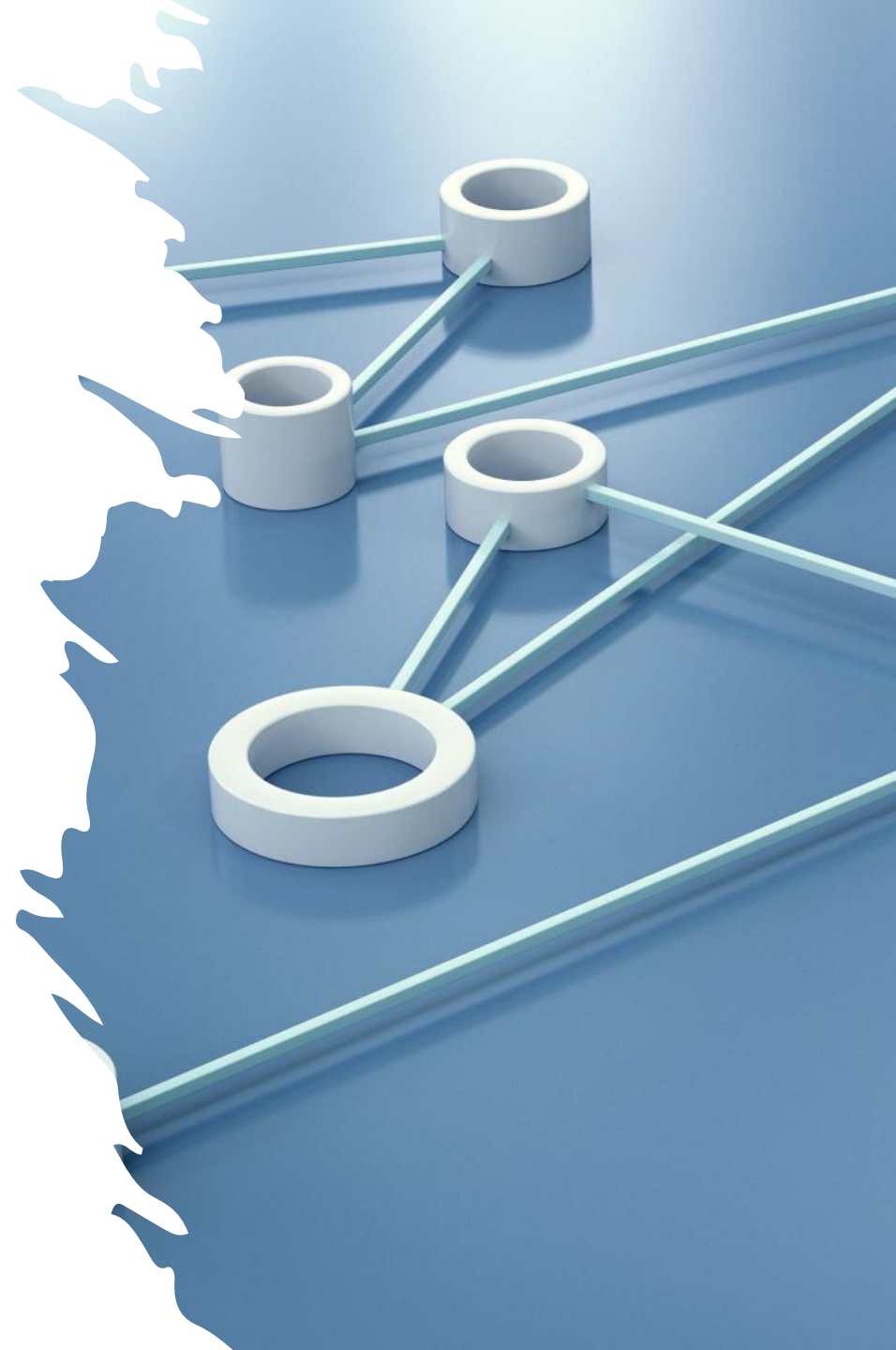


What is Signals in Angular?

- In Angular, Signals are a reactivity system introduced in Angular 16 (and improved in Angular 17) to make state management simpler, more predictable, and more efficient.

Definition

- A signal is a special kind of variable that:
- Holds a value (like a state variable).
- Notifies automatically when its value changes.
- Similar to state in React
- Updates the UI or dependent logic whenever it's read and modified.



What makes signals unique

1. Fine-Grained Reactivity

- Only the parts of the template or logic that depend on a signal are updated.
- Unlike Angular's traditional zone.js change detection, which re-checks the entire component tree.

👉 Example: If count changes, only the elements using count() are re-rendered — not the whole app.

2. Pull-Based (Synchronous)

- Signals are pull-based → You read the current value directly (count()).
- Observables are push-based → You “subscribe” to changes and wait for emissions.
- This makes signals simpler and predictable for local state.

3. Always Hold a Value

- A signal always has a current value (like React state).
- Observables don't always have a value until they emit.
- Promises resolve only once, not continuously.

What makes signal unique (Cont.)

4. Seamless Template Integration

- In Angular templates, you can call `{{ count() }}` directly — no need for async pipe.
- Removes extra boilerplate (`.subscribe()`, `.unsubscribe()`).

5. Automatic Dependency Tracking

- If a computed signal depends on other signals, Angular auto-tracks dependencies.
- No manual wiring or subscription logic needed.



Example:

- `count = signal(2);`
- `double = computed(() => this.count() * 2);`
- Whenever `count` changes, `double` auto-updates.

6. Side Effects with Control

- Signals use `effect()` to run side effects when values change (like logging, `localStorage` sync).
- Unlike RxJS, you don't need operators like `tap()` or manual `unsubscribe` logic.

7. Lightweight State Management

- Perfect for component-level state (like counters, toggles, form state).
- Can also scale to app-wide state with `readonly/writable` signals.
- Provides a simpler alternative to NgRx or services for smaller apps.

How to Use

-
- Create: `count = signal(0)`
 - Read: `count()`
 - Write: `count.set(5)`, `count.update(c => c+1)`

Computed Signals:

- `double = computed(() => count() * 2)`

Possible Patterns with Signals

Signal arrays & objects(Immutable Updates)

Writable vs Readonly signals

Using signals for global state

Signal Stores (Composable State Containers)

Combining Multiple Signals

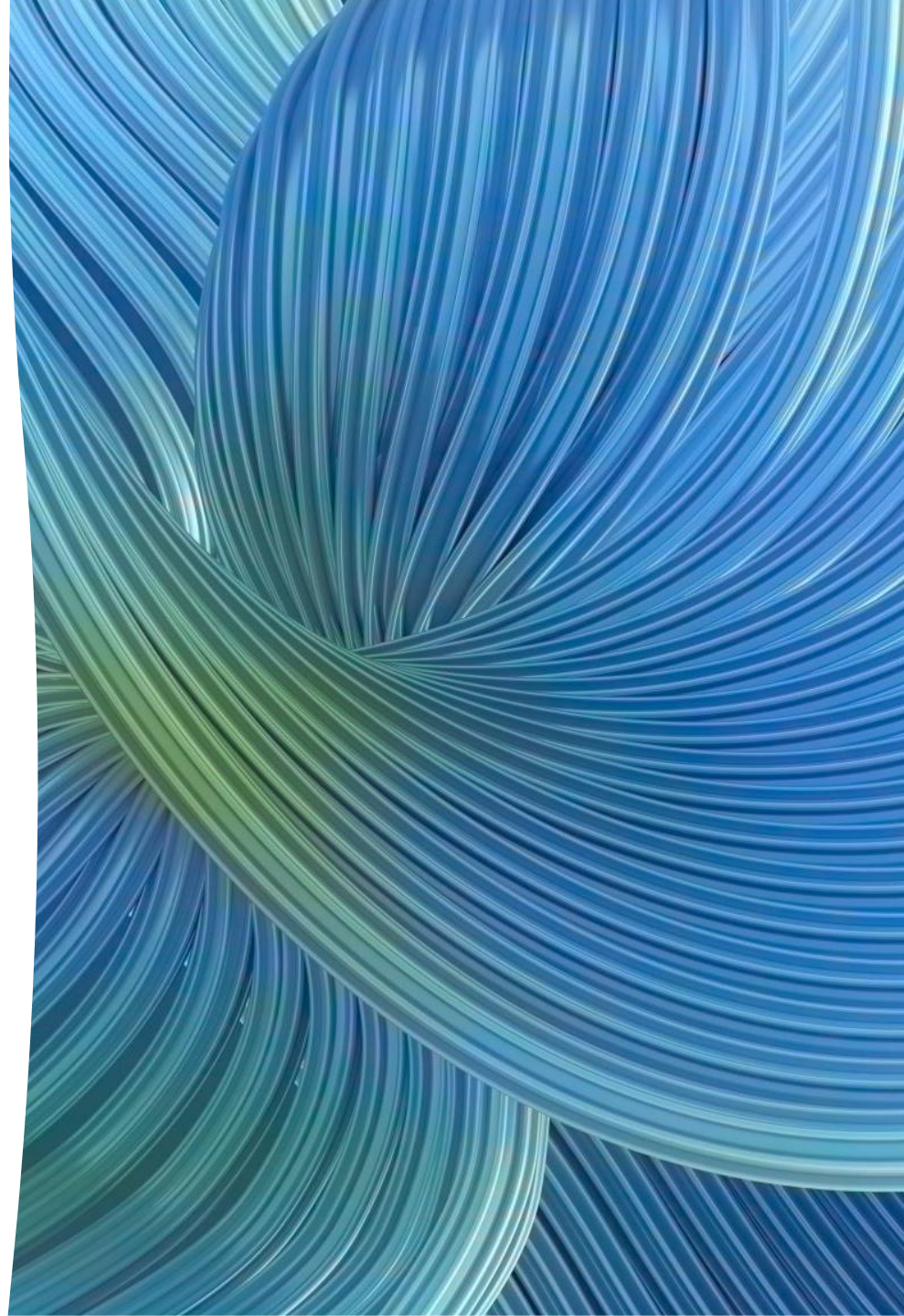
Signal-Based Selectors (Like NgRx Selectors)

Reactive Forms with Signals

Async Interop with RxJS

Effects with Cleanup & Controlled Writes

Computed Signals for Derived State



Running Side Effects with Signals

- Use `effect()` to react when signal changes

Example:

```
count = signal(0)
```

```
effect(() => console.log(`Count is: ${count()}`))
```

Note - Ideal for logging, HTTP calls, localStorage updates

Advantages of Signals



Fine-grained reactivity



Simplifies state management



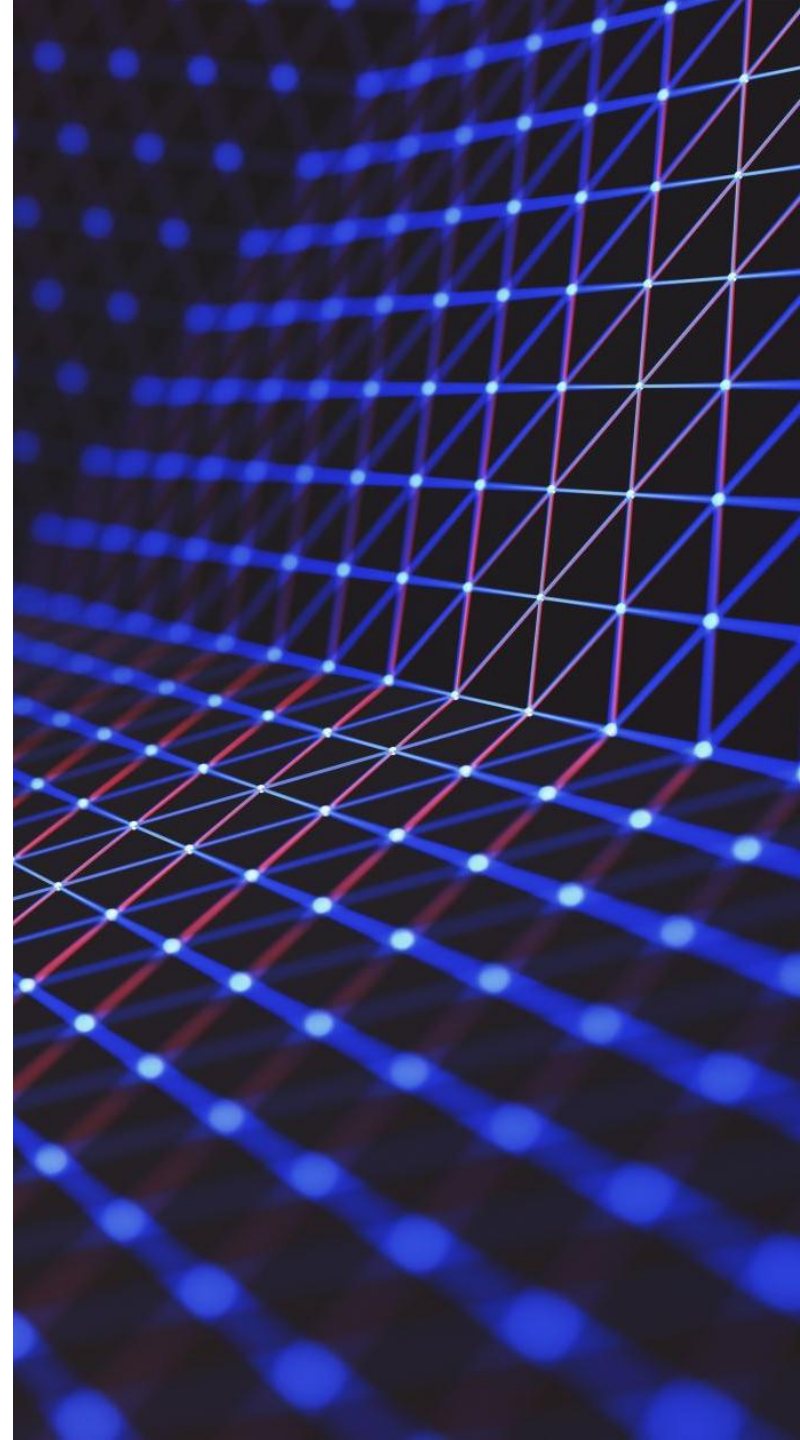
Eliminates manual subscriptions



Works seamlessly with Angular templates



Better performance than zone.js driven change detection



Signals vs Observables vs Promises

- Signals:
 - Holds current value
 - Pull-based, synchronous
- Observables:
 - Emits stream of values
 - Push-based, async or sync
- Promises:
 - One-time async value
 - Cannot emit multiple values

Common Pitfalls with Signals

- Forgetting to call signals with `()` in templates
- Using signals for async streams instead of Observables
- Circular dependencies in computed signals
- Overusing signals where simple variables suffice
- Not encapsulating writable signals (exposing `.set()` to components that shouldn't modify state).
- Running heavy logic inside `effect()` without cleanup → may cause performance issues.
- Confusing signals with Observables → signals are sync & stateful, not async streams.